

Mid-term Exam (Graph Neural Networks - Fall 2026)

해설지 ENG

Full Name:

Student ID:

Note: Students should write in English for evaluation.

1회차

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

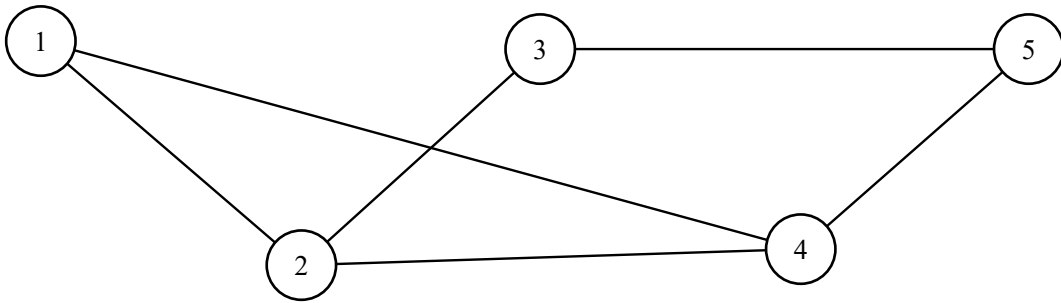


Figure. Graph G

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

Solution

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.
- $P(Y_1) = 1, P(Y_2) = 0.6667, P(Y_3) = 0.25, P(Y_4) = 0.5, P(Y_5) = 0$
- $P(Y_1) = 1, P(Y_2) = 0.5833, P(Y_3) = 0.3333, P(Y_4) = 0.5556, P(Y_5) = 0$

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ -2 \\ 1 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{C, D\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

Solution

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.
- $q(A) = 0.6237$, $q(B) = 0.0672$, $q(C) = 0.0672$, $q(D) = 0.2177$, $q(E) = 0.0242$
- $h^1(A) = 0$

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ -1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = E$.
- Calculate $E_{p(u_j | E)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(E)$.

Solution

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.
- $N(E) = 1$, $p(B|E) = 0.3333$, $p(C|E) = 0.3333$, $p(D|E) = 0.3333$
- $E = 1.3333$, $h_1(E) = 1.3333$

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

Solution

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.
- $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
 $Z1 = [0.4, 0.2, 0, 0.4]$
- $Z2 = [0.08, 0.52, 0.32, 0.08]$

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ 2 \\ 2 \end{bmatrix}$$

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.3$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

Solution

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to α times $(1-\alpha)$ to the power k , which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.
- $H_1 = [0, 3, 3, 0]$, $H_2 = [3, 0, 0, 3]$
- $Z = [2.2, -0.5, 2.3, 0.8]$

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -3 \\ 3 \\ -1 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

Solution

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.
- $A^0 H = [1, -3, 3, -1]$, $A^1 H = [-3, 1, -2, 0]$, $A^2 H = [1, -1.6667, 0.5, -0.5]$
- $H^1 =$ concatenation of the three blocks: $[1, 0, 3, 0] \parallel [0, 1, 0, 0] \parallel [1, 0, 0.5, 0]$
 Delta = $[-1, 1, -0.5, 0]$

7. (**PairNorm**) After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} 2 & -2 \\ 4 & -1 \\ 4 & 3 \\ 3 & 2 \end{bmatrix}$$

PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

Solution

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.
- Mean = [3.25, 0.5], centered = [-1.25, -2.5]; [0.75, -1.5]; [0.75, 2.5]; [-0.25, 1.5]
- Scaling denominator = 2.222, $x = [-0.5625, -1.1251]; [0.3375, -0.6751]; [0.3375, 1.1251]; [-0.1125, 0.6751]$

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -3 \\ 3 \\ 2 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (B, C) and (A, C) are dropped. Recalculate $H^{(1)}$.

Solution

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.
- Expected number of surviving edges = 4, $H1 = [0.6667, 2.6667, 0.6667, 1]$
- $H1 = [0, 2.5, 2, 1]$

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

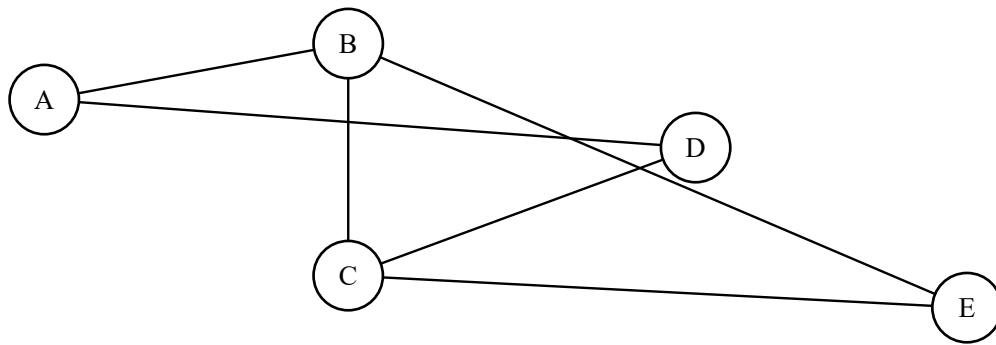


Figure 1. Graph G

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

Solution

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.
- $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow E) = 2$
- $P(A) = 0.1111$, $P(C) = 0.4444$, $P(E) = 0.4444$

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = 1, u'_B = 0.5, u'_C = -1, u'_D = -0.5.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate $p_2(A|C)$, $p_2(B|C)$, $p_2(C|C)$ and $p_2(D|C)$.

Solution

ENG)

- a) Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.
- b) $p_2(A|C) = 0.1412$, $p_2(B|C) = 0.1806$, $p_2(C|C) = 0.3819$, $p_2(D|C) = 0.2963$

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 0 \\ 2 \\ 3 \\ -2 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.
- $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0.5, 0]; [0.25, 0.25, 0, 0.25, 0.25]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 0, 1, 0, 0]$
- $H_1 = [2.5, 2.5, 0.75, 1.3333, 2]$

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

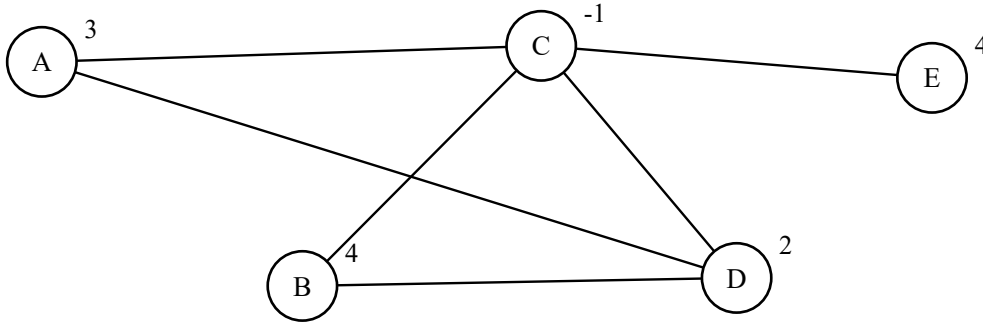


Figure. Graph G

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.
- $h(A) = [3, 0.5]$, $h(B) = [4, 0.5]$, $h(C) = [0, 3.25]$, $h(D) = [2, 2]$, $h(E) = [4, 0]$
- $h_G = [2.6, 1.25]$

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

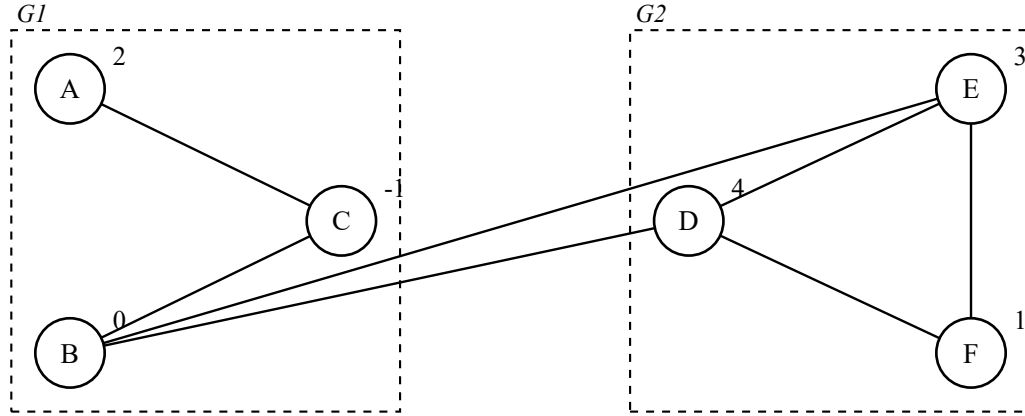


Figure. Graph G

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.
- $h(A) = [2, 0]$, $h(B) = [0, 0]$, $h(C) = [0, 1]$, $h(D) = [4, 2]$, $h(E) = [3, 2.5]$, $h(F) = [1, 3.5]$

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

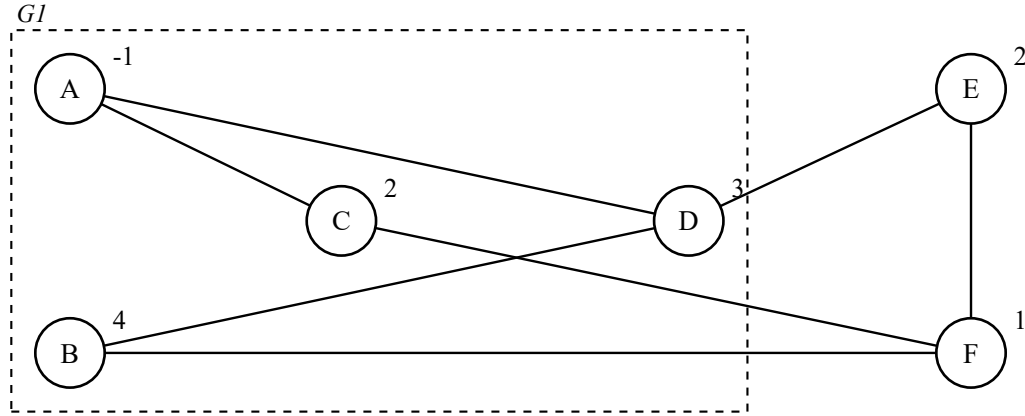


Figure. Graph G

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.
- $h(A) = [0, 2.5]$, $h(B) = [4, 3]$, $h(C) = [2, 0]$, $h(D) = [3, 1.5]$

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

Solution

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.
- Atilde = [0, 0.3333, 0.3333, 0.3333, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0.3333, 0.3333, 0, 0]; [0, 1, 0, 0, 0]
- H1 = [0.3333, 0.3333, 0, 0, 2]
- H2 = [0.3333, 0.7778, 0.1667, 0.2222, 2]

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -1 \\ 2 \\ -3 \\ 0 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.5$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.
- $\tilde{A} = [0, 0, 0.3333, 0.3333, 0.3333]; [0, 0, 0, 0, 1]; [0.5, 0, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$
- $H_1 = [0.1667, 0, 0.2, 0.3667, 0]$

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 3 \\ 3 \\ 1 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

Solution

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.
- $H1 = [1, 5, 4, 2.6667]$
 $H2 = [4.3333, 8.3333, 6.8889, 6]$
- Plain GCN without residual: $H1 = [2, 2, 1, 1.6667]$ (the node's own value and negatives disappear)

2회차

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1$, $P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

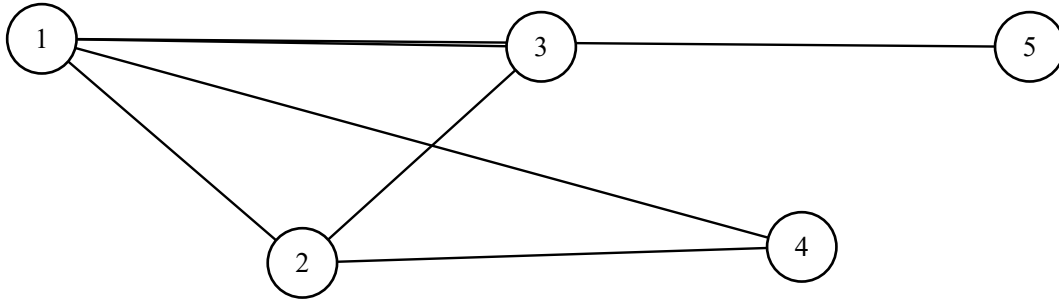


Figure. Graph G

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

Solution

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.
- $P(Y_1) = 1$, $P(Y_2) = 0.6667$, $P(Y_3) = 0.75$, $P(Y_4) = 0.75$, $P(Y_5) = 0$
- $P(Y_1) = 1$, $P(Y_2) = 0.8333$, $P(Y_3) = 0.8333$, $P(Y_4) = 0.8333$, $P(Y_5) = 0$

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ -2 \\ 3 \\ 1 \\ -1 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{D, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

Solution

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.
- $q(A) = 0.4889$, $q(B) = 0.1889$, $q(C) = 0.0444$, $q(D) = 0.1889$, $q(E) = 0.0889$
- $h_1(A) = 0.1667$

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 4 \\ -1 \\ 2 \\ 2 \\ 1 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = C$.
- Calculate $E_{p(u_j | C)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(C)$.

Solution

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.
- $N(C) = 1$, $p(A|C) = 0.3333$, $p(D|C) = 0.3333$, $p(E|C) = 0.3333$
- $E = 2.3333$, $h_1(C) = 2.3333$

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.5$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

Solution

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.
- $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0.5, 0]; [0.3333, 0.3333, 0, 0.3333]; [0, 0, 1, 0]$
 $Z^1 = [0, 0, 0.1667, 0.5]$
- $Z^2 = [0.0417, 0.0417, 0.0833, 0.5833]$

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 0 \\ -1 \\ -2 \end{bmatrix}$$

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.3$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

Solution

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to α times $(1-\alpha)$ to the power k , which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.
- $H_1 = [-1.5, -2, 0.5, 0.6667]$, $H_2 = [0.5833, 0.6667, -0.4167, -1]$
- $Z = [1.5333, -0.7333, -0.4667, -1]$

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ -1 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

Solution

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.
- $A^0 H = [2, 3, 4, -1]$, $A^1 H = [1, 0.5, -1, 3]$, $A^2 H = [1.75, 2, 3, 0.1667]$
- $H^1 =$ concatenation of the three blocks: $[2, 3, 4, 0] \parallel [1, 0.5, 0, 3] \parallel [1.75, 2, 3, 0.1667]$
 Delta = $[-0.75, -1.5, -3, 2.8333]$

7. (**PairNorm**) After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} 0 & 4 \\ 1 & 3 \\ 1 & -2 \\ -2 & 1 \end{bmatrix}$$

PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

Solution

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.
- Mean = [0, 1.5], centered = [0, 2.5]; [1, 1.5]; [1, -3.5]; [-2, -0.5]
- Scaling denominator = 2.5981, $x = [0, 0.9623]; [0.3849, 0.5774]; [0.3849, -1.3472]; [-0.7698, -0.1925]$

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -2 \\ 2 \\ 2 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (C, D) and (A, C) are dropped. Recalculate $H^{(1)}$.

Solution

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.
- Expected number of surviving edges = 4, $H1 = [0.6667, 1.3333, 0, 0]$
- $H1 = [0, 1.3333, 0, 0]$

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

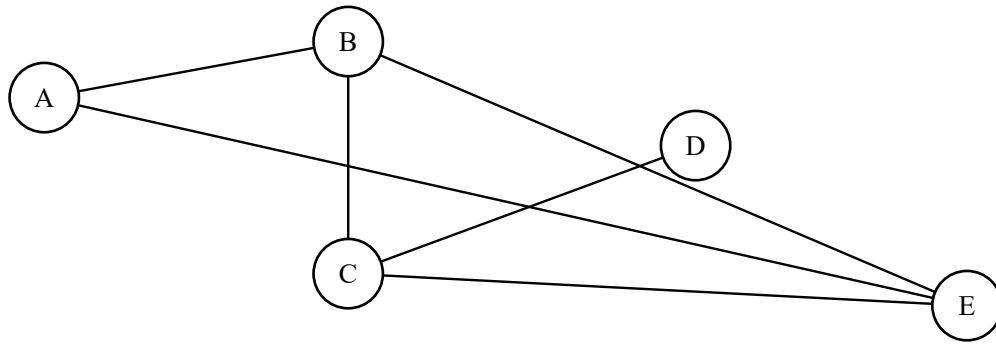


Figure 1. Graph G

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

Solution

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.
- $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow E) = 1$
- $P(A) = 0.1429$, $P(C) = 0.5714$, $P(E) = 0.2857$

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = 1, u'_C = 0.5, u'_D = -1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate $p_2(A|C)$, $p_2(B|C)$, $p_2(C|C)$ and $p_2(D|C)$.

Solution

ENG)

- a) Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.
- b) $p_2(A|C) = 0.2963$, $p_2(B|C) = 0.1412$, $p_2(C|C) = 0.1806$, $p_2(D|C) = 0.3819$

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ -2 \\ 2 \\ -2 \\ 4 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.
- $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 0.5, 0, 0.5]; [0, 0.3333, 0.3333, 0.3333, 0]$
- $H_1 = [0, 1.5, 0.3333, 3, 0]$

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

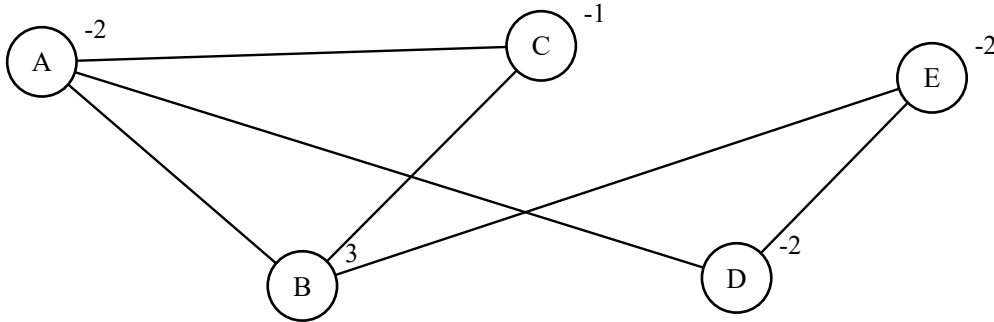


Figure. Graph G

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.
- $h(A) = [0, 0]$, $h(B) = [3, 0]$, $h(C) = [0, 0.5]$, $h(D) = [0, 0]$, $h(E) = [0, 0.5]$
- $h_G = [0.6, 0.2]$

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

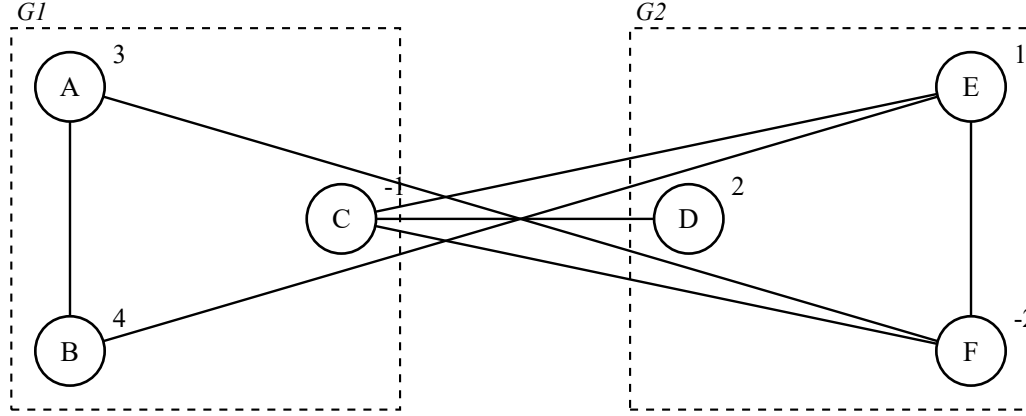


Figure. Graph G

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.
- $h(A) = [3, 4]$, $h(B) = [4, 3]$, $h(C) = [0, 0]$, $h(D) = [2, 0]$, $h(E) = [1, 0]$, $h(F) = [0, 1]$

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

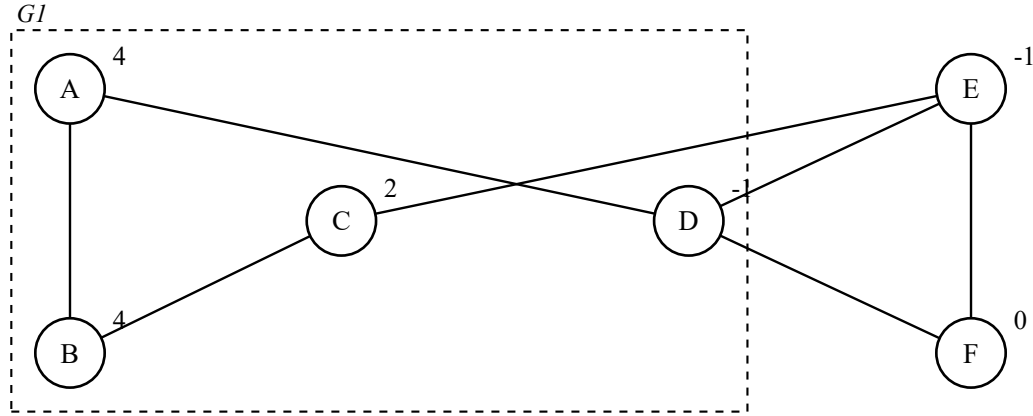


Figure. Graph G

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.
- $h(A) = [4, 1.5]$, $h(B) = [4, 3]$, $h(C) = [2, 4]$, $h(D) = [0, 4]$

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 0 \\ -3 \\ -1 \\ -1 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

Solution

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.
- $A_{tilde} = [0, 0.3333, 0, 0.3333, 0.3333]; [0.5, 0, 0, 0.5, 0]; [0, 0, 0, 0, 1]; [0.3333, 0.3333, 0, 0, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]$
 $H_1 = [0, 1, 0, 0.6667, 0]$
- $H_2 = [0.5556, 1, 0, 0.6667, 0.2222]$

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -3 \\ -1 \\ 1 \\ -1 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.
- $\tilde{A} = [0, 0, 0, 0.5, 0.5]; [0, 0, 0.5, 0, 0.5]; [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0.3333, 0.3333, 0, 0.3333, 0]$
- $H_1 = [0, 0, 0, 0, 0]$

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 0 \\ 3 \\ 1 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

Solution

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.
- $H1 = [-1.5, 0.6667, 3.5, 1.3333]$
 $H2 = [-0.5, 1.7778, 4.5, 2.2222]$
- Plain GCN without residual: $H1 = [0.5, 0.6667, 0.5, 0.3333]$ (the node's own value and negatives disappear)

3회차

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1$, $P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

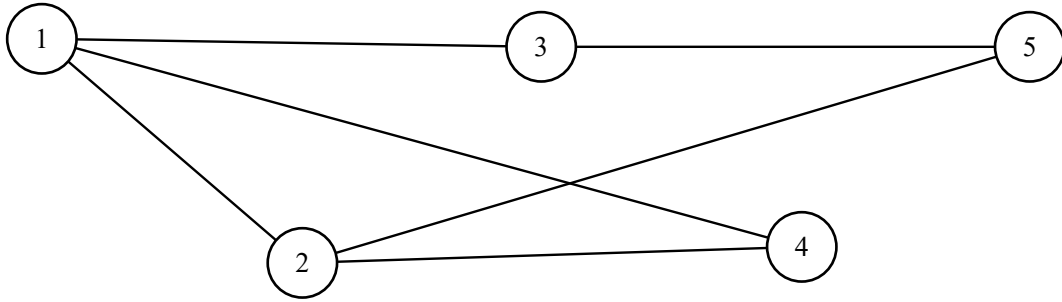


Figure. Graph G

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

Solution

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.
- $P(Y_1) = 1$, $P(Y_2) = 0.5$, $P(Y_3) = 0.5$, $P(Y_4) = 0.75$, $P(Y_5) = 0$
- $P(Y_1) = 1$, $P(Y_2) = 0.5833$, $P(Y_3) = 0.5$, $P(Y_4) = 0.75$, $P(Y_5) = 0$

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 1 \\ 0 \\ -2 \\ -1 \\ 2 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{B, C\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

Solution

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.
- $q(A) = 0.0444$, $q(B) = 0.4889$, $q(C) = 0.0889$, $q(D) = 0.1889$, $q(E) = 0.1889$
- $h^{(1)}(A) = 0$

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -1 \\ -1 \\ 4 \\ 1 \\ -2 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = C$.
- Calculate $E_{p(u_j | C)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(C)$.

Solution

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.
- $N(C) = 1$, $p(A|C) = 0.3333$, $p(D|C) = 0.3333$, $p(E|C) = 0.3333$
- $E = -0.6667$, $h_1(C) = 0$

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.5$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

Solution

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.
- $\tilde{A} = [0, 0, 0.5, 0.5]; [0, 0, 0.5, 0.5]; [0.5, 0.5, 0, 0]; [0.5, 0.5, 0, 0]$
 $Z_1 = [0.25, 0.25, 0, 0.5]$
- $Z_2 = [0.125, 0.125, 0.125, 0.625]$

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ -1 \\ 4 \end{bmatrix}$$

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.5$, $\gamma_1 = 0.2$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

Solution

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to alpha times (1-alpha) to the power k, which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.
- $H_1 = [1, 1.5, 1, 1.5]$, $H_2 = [1.5, 1, 1.5, 1]$
- $Z = [1.9, -0.9, -0.6, 2.1]$

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ -2 \\ 1 \\ -1 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

Solution

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power 1 times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.
- $A^0 H = [2, -2, 1, -1]$, $A^1 H = [0, -1, 0.5, 0.3333]$, $A^2 H = [0.4167, 0.3333, 0.1667, -0.1667]$
- $H^1 =$ concatenation of the three blocks: $[2, 0, 1, 0] \parallel [0, 0, 0.5, 0.3333] \parallel [0.4167, 0.3333, 0.1667, 0]$
 Delta = $[-0.4167, -0.3333, 0.3333, 0.3333]$

7. (**PairNorm**) After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} 4 & 0 \\ 1 & 3 \\ 1 & -1 \\ -1 & 4 \end{bmatrix}$$

PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

Solution

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.
- Mean = [1.25, 1.5], centered = [2.75, -1.5]; [-0.25, 1.5]; [-0.25, -2.5]; [-2.25, 2.5]
- Scaling denominator = 2.7272, $x = [1.0084, -0.55]; [-0.0917, 0.55]; [-0.0917, -0.9167]; [-0.825, 0.9167]$

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ -3 \\ 4 \\ 3 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (A, C) and (B, D) are dropped. Recalculate $H^{(1)}$.

Solution

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.
- Expected number of surviving edges = 4, $H1 = [1.3333, 2.3333, 0, 0.3333]$
- $H1 = [0, 2, 0, 2]$

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

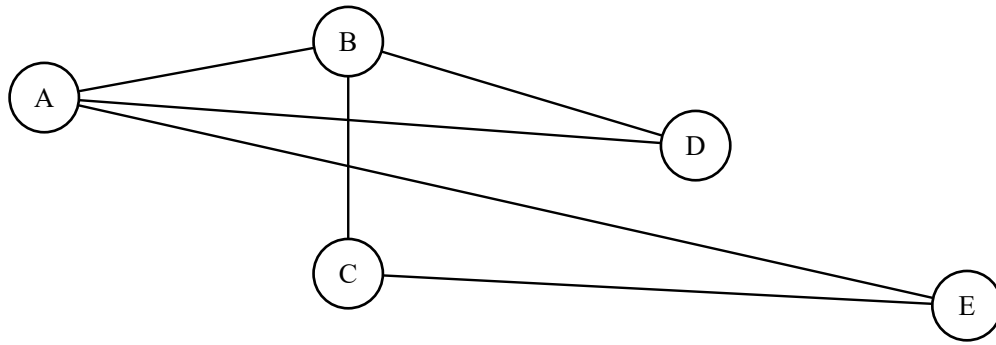


Figure 1. Graph G

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

Solution

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.
- $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 2$, $\pi(B \rightarrow D) = 1$
- $P(A) = 0.1429$, $P(C) = 0.5714$, $P(D) = 0.2857$

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = 0.5, u'_C = 1, u'_D = -1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

Solution

ENG)

- a) Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.
- b) $p_2(A|B) = 0.1806$, $p_2(B|B) = 0.2963$, $p_2(C|B) = 0.3819$, $p_2(D|B) = 0.1412$

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -2 \\ -1 \\ 4 \\ 4 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.
- $\tilde{A} = [0, 0, 0.5, 0, 0.5]; [0, 0, 0.3333, 0.3333, 0.3333]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 1, 0, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$
- $H_1 = [1.5, 2.3333, 1, 0, 0]$

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

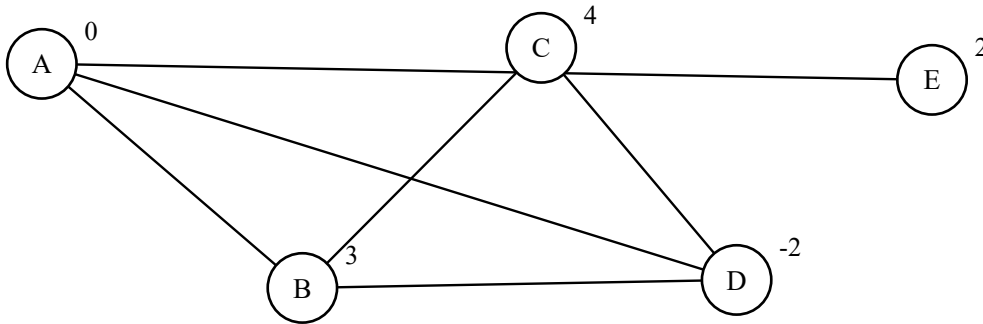


Figure. Graph G

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.
- $h(A) = [0, 1]$, $h(B) = [3, 0.6667]$, $h(C) = [4, 0.5]$, $h(D) = [0, 2.3333]$, $h(E) = [2, 0]$
- $h_G = [1.8, 0.9]$

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

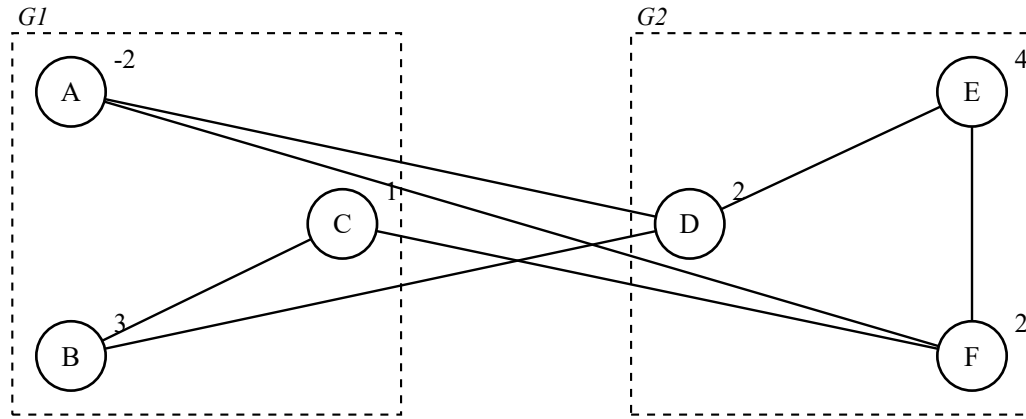


Figure. Graph G

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.
- $h(A) = [0, 0]$, $h(B) = [3, 1]$, $h(C) = [1, 3]$, $h(D) = [2, 4]$, $h(E) = [4, 2]$, $h(F) = [2, 4]$

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

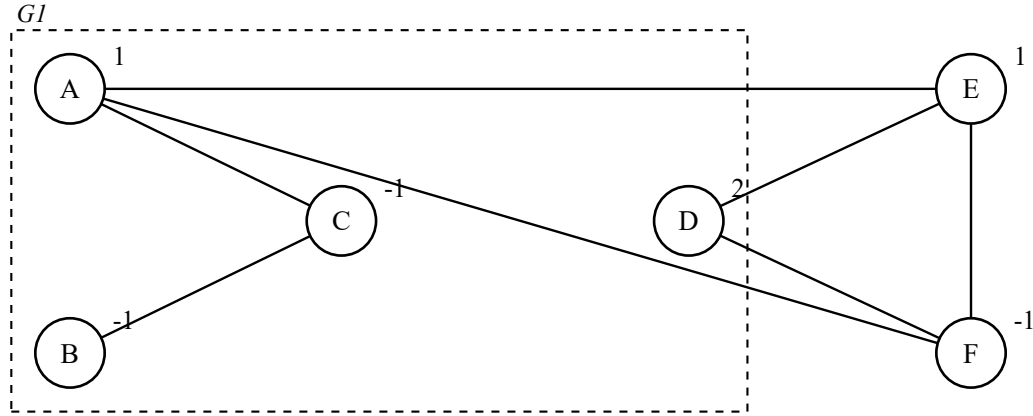


Figure. Graph G

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.
- $h(A) = [1, 0]$, $h(B) = [0, 0]$, $h(C) = [0, 0]$, $h(D) = [2, 0]$

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -3 \\ -3 \\ -3 \\ 3 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

Solution

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.
- $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.3333, 0, 0, 0.3333, 0.3333]; [1, 0, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]; [0.5, 0.5, 0, 0, 0]$
- $H_1 = [0, 1.3333, 4, 0.5, 0.5]$
- $H_2 = [1.5833, 1.3333, 4, 0.6667, 0.6667]$

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 3 \\ 3 \\ 0 \\ -1 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.2$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.
- $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$
- $H_1 = [1.08, 0, 0, 1.44, 1.32]$

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

Solution

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.
- $H1 = [-1.5, 1, 2, 1.5]$
 $H2 = [0, 1.6667, 2.3333, 3]$
- Plain GCN without residual: $H1 = [1.5, 0, 0, 1.5]$ (the node's own value and negatives disappear)

4회차

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1, P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

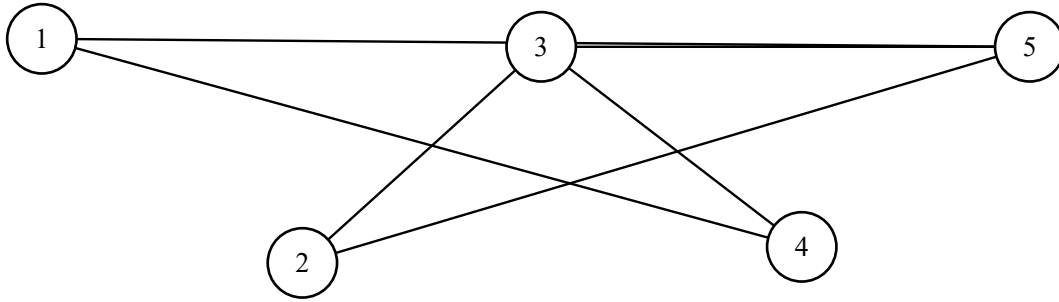


Figure. Graph G

- a) Explain the key assumption behind relational classification, and state one limitation.
- b) Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- c) Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

Solution

ENG)

- a) Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.
- b) $P(Y_1) = 1, P(Y_2) = 0.25, P(Y_3) = 0.3333, P(Y_4) = 0.75, P(Y_5) = 0$
- c) $P(Y_1) = 1, P(Y_2) = 0.1667, P(Y_3) = 0.3333, P(Y_4) = 0.6667, P(Y_5) = 0$

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 3 \\ 3 \\ 2 \\ 0 \\ 2 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{D, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

Solution

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.
- $q(A) = 0.2821$, $q(B) = 0.1667$, $q(C) = 0.1667$, $q(D) = 0.2821$, $q(E) = 0.1026$
- $h_1(A) = 0.3333$

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 0 \\ -1 \\ 2 \\ 4 \\ 2 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = B$.
- Calculate $E_{p(u_j | B)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(B)$.

Solution

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.
- $N(B) = 1$, $p(A|B) = 0.3333$, $p(C|B) = 0.3333$, $p(D|B) = 0.3333$
- $E = 2$, $h_1(B) = 2$

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

Solution

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.
- $\tilde{A} = [0, 0.3333, 0.3333, 0.3333]; [1, 0, 0, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0.5, 0]$
 $Z^1 = [0.2, 0.8, 0.4, 0.4]$
- $Z^2 = [0.6267, 0.16, 0.24, 0.24]$

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 3 \\ -2 \\ 3 \end{bmatrix}$$

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.5$, $\gamma_1 = 0.4$, $\gamma_2 = -0.2$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

Solution

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to α times $(1-\alpha)$ to the power k , which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.
- $H_1 = [0.5, 0, 1, 3]$, $H_2 = [0.5, 1.5, 0.25, 0]$
- $Z = [-0.4, 1.2, -0.65, 2.7]$

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ -2 \\ -2 \\ 3 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A}H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

Solution

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.
- $A^0H = [-2, -2, -2, 3]$, $A^1H = [-2, 0.5, 0.5, -2]$, $A^2H = [0.5, -2, -2, 0.5]$
- $H^1 =$ concatenation of the three blocks: $[0, 0, 0, 3] \parallel [0, 0.5, 0.5, 0] \parallel [0.5, 0, 0, 0.5]$
 Delta = $[-0.5, 0.5, 0.5, -0.5]$

7. (**PairNorm**) After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 4 & 0 \\ -2 & 0 \end{bmatrix}$$

PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

Solution

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.
- Mean = [0.25, 0], centered = [-1.25, 0]; [-0.25, 0]; [3.75, 0]; [-2.25, 0]
- Scaling denominator = 2.2776, $x = [-0.5488, 0]; [-0.1098, 0]; [1.6465, 0]; [-0.9879, 0]$

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 0 \\ 3 \\ 3 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (A, D) and (A, C) are dropped. Recalculate $H^{(1)}$.

Solution

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.
- Expected number of surviving edges = 4, $H_1 = [2, 1.6667, 0.6667, 0.6667]$
- $H_1 = [0, 1.6667, 1.5, 1.5]$

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

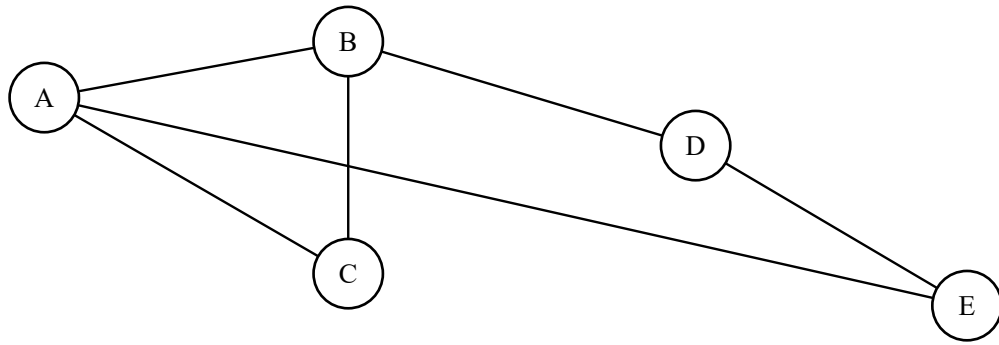


Figure 1. Graph G

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

Solution

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.
- $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 1$, $\pi(B \rightarrow D) = 2$
- $P(A) = 0.1429$, $P(C) = 0.2857$, $P(D) = 0.5714$

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = -0.5, u'_B = -1, u'_C = 0.5, u'_D = 1.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

Solution

ENG)

- a) Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.
- b) $p_2(A|B) = 0.1806$, $p_2(B|B) = 0.1412$, $p_2(C|B) = 0.2963$, $p_2(D|B) = 0.3819$

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ -3 \\ -2 \\ 2 \\ -2 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.
- $\tilde{A} = [0, 0.25, 0.25, 0.25, 0.25]; [0.5, 0, 0, 0, 0.5]; [0.5, 0, 0, 0.5, 0]; [0.5, 0, 0.5, 0, 0]; [0.5, 0.5, 0, 0, 0]$
- $H_1 = [0, 0, 0.5, 0, 0]$

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

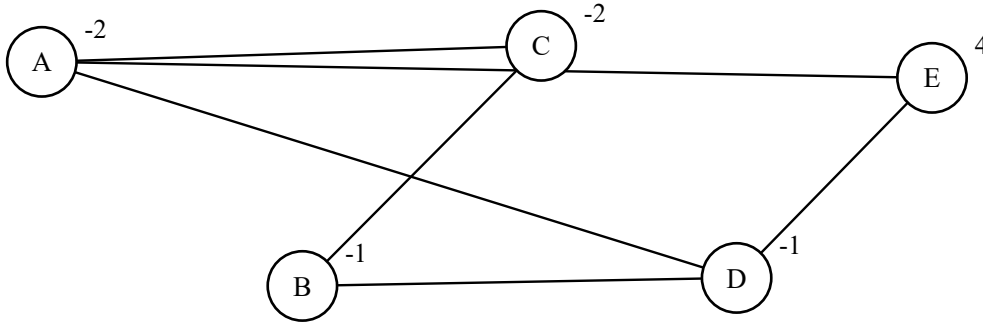


Figure. Graph G

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.
- $h(A) = [0, 0.3333]$, $h(B) = [0, 0]$, $h(C) = [0, 0]$, $h(D) = [0, 0.3333]$, $h(E) = [4, 0]$
- $h_G = [0.8, 0.1333]$

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

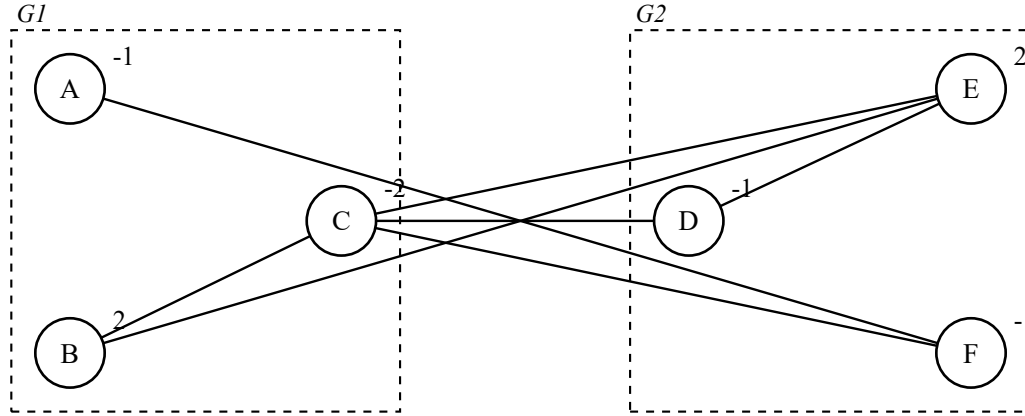


Figure. Graph G

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.
- $h(A) = [0, 0]$, $h(B) = [2, 0]$, $h(C) = [0, 2]$, $h(D) = [0, 2]$, $h(E) = [2, 0]$, $h(F) = [0, 0]$

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

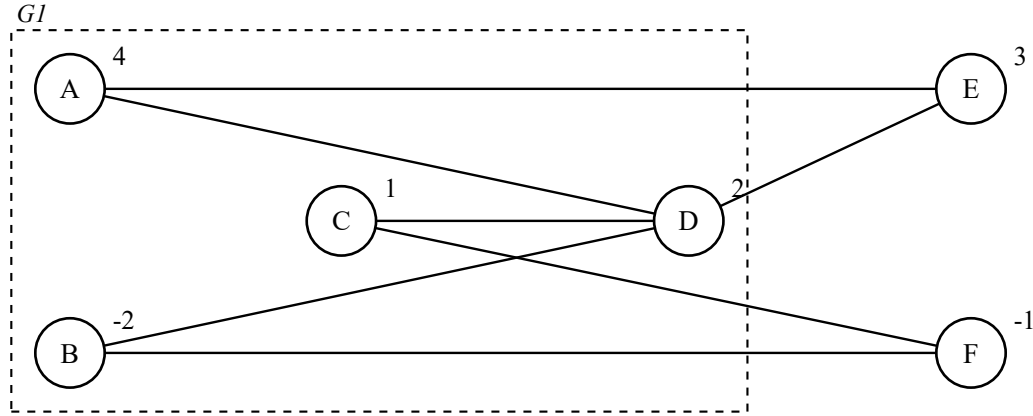


Figure. Graph G

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.
- $h(A) = [4, 2]$, $h(B) = [0, 2]$, $h(C) = [1, 2]$, $h(D) = [2, 1]$

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 0 \\ 0 \\ -2 \\ 3 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

Solution

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.
- Atilde = [0, 0.3333, 0, 0.3333, 0.3333]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.5, 0.5, 0, 0, 0]; [0.5, 0, 0.5, 0, 0]
- H1 = [0.3333, 0, 1.5, 1, 1]
- H2 = [0.6667, 0.9444, 1.5, 1, 1]

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ 4 \\ -2 \\ 3 \\ -1 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.5$, $\beta = 0.5$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.
- $\tilde{A} = [0, 0.5, 0, 0.5, 0]; [0.3333, 0, 0.3333, 0.3333, 0]; [0, 0.5, 0, 0, 0.5]; [0.3333, 0.3333, 0, 0, 0.3333]; [0, 0, 0.5, 0.5, 0]$
- $H_1 = [1.625, 1.3333, 0, 1.25, 0]$

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 4 \\ -2 \\ 0 \\ -2 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

Solution

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.
- $H1 = [4, -1.3333, 0, -1.3333]$
 $H2 = [4, -0.4444, 0, -0.4444]$
- Plain GCN without residual: $H1 = [0, 0.6667, 0, 0.6667]$ (the node's own value and negatives disappear)

5회차

1. (Relational Classification) Consider an undirected graph G of five nodes given in the following figure. Nodes 1 and 5 are labelled ($P(Y_1) = 1$, $P(Y_5) = 0$) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = (\sum_{(v,u) \in E} A_{v,u} P(Y_u = c)) / (\sum_{(v,u) \in E} A_{v,u})$$

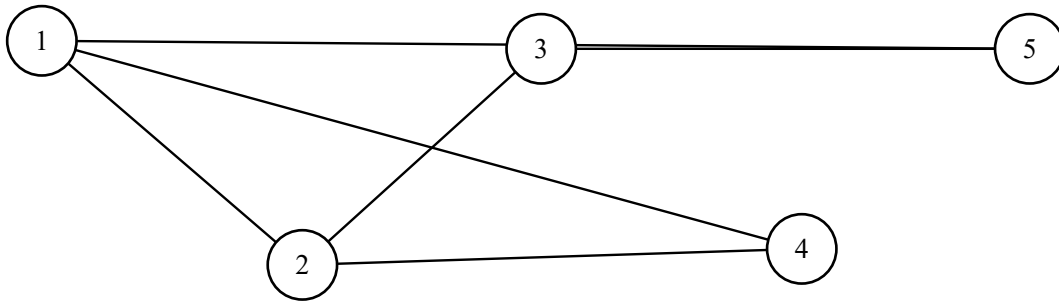


Figure. Graph G

- Explain the key assumption behind relational classification, and state one limitation.
- Calculate $P(Y_v)$ for every unlabelled node after the 1st iteration.
- Calculate $P(Y_v)$ after the 2nd iteration, using the previous iteration's values only.

Solution

ENG)

- Write these three sentences.
 - Relational classification assumes homophily: connected nodes tend to share the same label.
 - It repeatedly sets each unlabelled node to the weighted average of its neighbours' class probabilities.
 - Its limitation is that it uses only the graph structure and ignores node features.
- $P(Y_1) = 1$, $P(Y_2) = 0.6667$, $P(Y_3) = 0.25$, $P(Y_4) = 0.75$, $P(Y_5) = 0$
- $P(Y_1) = 1$, $P(Y_2) = 0.6667$, $P(Y_3) = 0.3333$, $P(Y_4) = 0.8333$, $P(Y_5) = 0$

2. (FastGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} -2 \\ 4 \\ 4 \\ -1 \\ -2 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where $\tilde{A} = D^{-1}A$ and the t_l nodes are drawn once per layer from $q(u) \propto \|\tilde{A}(:, u)\|^2$, with $\text{ReLU}(x) = \max(0, x)$.

- How does layer-wise sampling differ from the node-wise sampling of GraphSAGE?
- Calculate the importance distribution $q(u)$ over the five nodes.
- Suppose the sampled set is $\{B, E\}$ ($t_l = 2$). Calculate the estimate of $h^{(1)}(A)$.

Solution

ENG)

- Write these three sentences.
 - In node-wise sampling, each node samples its own neighbours, so the cost grows exponentially with depth.
 - FastGCN samples a fixed-size set of nodes once per layer, shared by all nodes in that layer.
 - Therefore the cost grows only linearly with depth instead of exponentially.
- $q(A) = 0.0889$, $q(B) = 0.0444$, $q(C) = 0.1889$, $q(D) = 0.1889$, $q(E) = 0.4889$
- $h^1(A) = 0$

3. (ASGCN) Consider the undirected graph G with adjacency matrix A and node feature vector $h^{(0)}$ below. ASGCN rewrites a GCN layer as an expectation:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 4 \\ 4 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma(W^{(l)} N(v_i) E_{p(u_j | v_i)}[h^{(l)}(u_j)])$$

where $\tilde{A} = D^{-1}A$, $N(v_i) = \sum_j \tilde{A}_{ij}$, $p(u_j | v_i) = \tilde{A}_{ij} / N(v_i)$, $W^{(l)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- What does the proposal distribution depend on in ASGCN that it does not in FastGCN?
- Calculate $N(v_i)$ and $p(u_j | v_i)$ for the parent node $v_i = E$.
- Calculate $E_{p(u_j | E)}[h^{(0)}(u_j)]$ and the resulting $h^{(1)}(E)$.

Solution

ENG)

- Write these three sentences.
 - In FastGCN the sampling distribution is fixed in advance and does not depend on the parent nodes.
 - In ASGCN the proposal distribution is conditioned on the nodes of the upper layer.
 - This makes almost every sampled node useful and reduces the variance of the estimate.
- $N(E) = 1$, $p(A|E) = 0.5$, $p(D|E) = 0.5$
- $E = 3$, $h_1(E) = 3$

4. (APPNP) Given a graph with an adjacency matrix A and a prediction matrix H produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

APPNP propagates the predictions with a teleport probability α :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where $\tilde{A} = D^{-1}A$ and $\alpha = 0.2$. No nonlinearity is applied between steps.

- Explain the teleport mechanism and why it prevents over-smoothing.
- Calculate $\tilde{A}$ and $Z^{(1)}$.
- Calculate $Z^{(2)}$.

Solution

ENG)

- Write these three sentences.
 - At every propagation step the walk returns to the root node with probability alpha. This is the teleport mechanism.
 - Because a fixed fraction of the mass is pulled back at every step, the distribution stays local.
 - So we can use many propagation steps without over-smoothing, and propagation has no extra parameters.
- $\tilde{A} = [0, 0.5, 0.5, 0]; [0.5, 0, 0, 0.5]; [0.5, 0, 0, 0.5]; [0, 0.5, 0.5, 0]$
 $Z1 = [0.4, 0.2, 0, 0.4]$
- $Z2 = [0.08, 0.52, 0.32, 0.08]$

5. (GPR-GNN) Given a graph with an adjacency matrix A and hidden features $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 1 \\ -2 \\ 2 \end{bmatrix}$$

GPR-GNN separates feature extraction from topological propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where $\tilde{A} = D^{-1}A$, $K = 2$, $\gamma_0 = 0.7$, $\gamma_1 = 0.2$, $\gamma_2 = -0.3$.

- What can GPR-GNN do that APPNP cannot, and why do negative γ_k matter?
- Calculate $H^{(1)}$ and $H^{(2)}$.
- Calculate Z .

Solution

ENG)

- Write these three sentences.
 - APPNP fixes the coefficients to α times $(1-\alpha)$ to the power k , which are always positive and decreasing.
 - GPR-GNN learns the weights, and they are allowed to be negative.
 - A negative weight subtracts that hop, so the model can act as a high-pass filter on heterophilic graphs.
- $H_1 = [-0.5, -1.5, 0.6667, -2]$, $H_2 = [-0.4167, 0.0833, -1.3333, 0.6667]$
- $Z = [-0.675, 0.375, -0.8667, 0.8]$

6. (MixHop) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -3 \\ 1 \\ 0 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \parallel_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where $P = \{0, 1, 2\}$, $\hat{A} = D^{-1}A$, $W_j^{(i)} = 1$ and $\text{ReLU}(x) = \max(0, x)$.

- State Theorem 1 of MixHop about the two-hop Delta Operator.
- Calculate $\hat{A}^0 H^{(0)}$, $\hat{A}^1 H^{(0)}$ and $\hat{A}^2 H^{(0)}$.
- Calculate $H^{(1)}$ and the Delta Operator $\sigma(\hat{A} H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$.

Solution

ENG)

- Write these three sentences.
 - Theorem 1 says that a vanilla GCN cannot represent the two-hop Delta Operator, even with infinitely many layers.
 - Stacking GCN layers collapses to $\hat{A}$ to the power l times W , so the intermediate hops are never kept separately.
 - MixHop computes each power separately and concatenates them, so a later layer can subtract one from another.
- $A0H = [3, -3, 1, 0]$, $A1H = [-1, 1.5, 1.5, -1]$, $A2H = [1.5, -1, -1, 1.5]$
- $H1 =$ concatenation of the three blocks: $[3, 0, 1, 0] \parallel [0, 1.5, 1.5, 0] \parallel [1.5, 0, 0, 1.5]$
 Delta = $[-1.5, 1.5, 1.5, -1.5]$

7. (**PairNorm**) After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} -2 & 4 \\ 1 & -1 \\ -1 & 1 \\ -1 & 3 \end{bmatrix}$$

PairNorm normalizes them with a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_i x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_i \|x_i^c\|_2^2} \quad (\text{Scale})$$

with $s = 1$ and $n = 4$.

- Explain node-wise and feature-wise over-smoothing, and what PairNorm keeps constant.
- Calculate the centered features.
- Calculate the scaling denominator and the final x_i .

Solution

ENG)

- Write these three sentences.
 - Node-wise over-smoothing means the features of connected nodes become too similar.
 - Feature-wise over-smoothing means all node features become indistinguishable across the whole graph.
 - PairNorm keeps the total pairwise squared distance between node representations constant across layers.
- Mean = $[-0.75, 1.75]$, centered = $[-1.25, 2.25]$; $[1.75, -2.75]$; $[-0.25, -0.75]$; $[-0.25, 1.25]$
- Scaling denominator = 2.2079, $x = [-0.5661, 1.019]$; $[0.7926, -1.2455]$; $[-0.1132, -0.3397]$; $[-0.1132, 0.5661]$

8. (DropEdge) Consider the complete graph K_4 on nodes A, B, C and D with the adjacency matrix A and initial node feature matrix $H^{(0)}$ below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -2 \\ 2 \\ 1 \\ 0 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability $1 - p$ at every epoch:

$$H^{(1)} = \sigma((D^{(t)})^{-1} A^{(t)} H^{(0)})$$

where $A^{(t)}$ and $D^{(t)}$ are rebuilt from the surviving edges only, $p = 1/3$, and $\text{ReLU}(x) = \max(0, x)$.

- Which two problems does DropEdge target, and how does it differ from Dropout?
- Calculate the expected number of surviving edges, and $H^{(1)}$ on the full graph.
- Suppose the edges (C, D) and (A, D) are dropped. Recalculate $H^{(1)}$.

Solution

ENG)

- Write these three sentences.
 - DropEdge targets over-fitting and over-smoothing.
 - A different random subgraph is used at every epoch, which works as data augmentation on the graph.
 - Dropout zeroes entries of the feature tensor, while DropEdge removes edges of the adjacency matrix.
- Expected number of surviving edges = 4, $H1 = [1, 0, 0, 0.3333]$
- $H1 = [1.5, 0, 0, 2]$

9. (Node2vec) Consider an undirected graph G given in the following figure. A random walk has just moved from node A to node B , with $p = 2$ and $q = 0.5$.

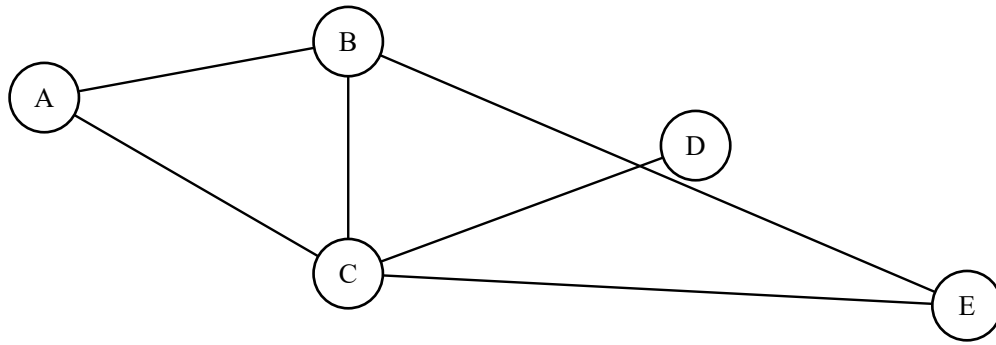


Figure 1. Graph G

The unnormalized transition probability from B to a next node x is $\pi_{Bx} = \alpha_{pq}(A, x)$, where $\alpha = 1/p$ if $d(A, x) = 0$, $\alpha = 1$ if $d(A, x) = 1$, and $\alpha = 1/q$ if $d(A, x) = 2$.

- Explain BFS and DFS, and explain which one a small q makes the walk behave like.
- List all possible next nodes from node B and calculate π_{Bx} .
- Calculate the normalized transition probabilities.

Solution

ENG)

- Write these three sentences.
 - BFS explores the immediate neighbourhood first, so the embedding captures the structural role of a node.
 - DFS moves far away from the source, so the embedding captures community membership.
 - A small q makes $1/q$ large, so the walk moves outward and behaves like DFS.
- $\pi(B \rightarrow A) = 0.5$, $\pi(B \rightarrow C) = 1$, $\pi(B \rightarrow E) = 2$
- $P(A) = 0.1429$, $P(C) = 0.2857$, $P(E) = 0.5714$

10. (LINE) Consider an undirected graph G with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = 0.5, u'_B = 1, u'_C = -1, u'_D = -0.5.$$

The second-order proximity is $p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x)$. Given $e^{0.25} = 1.28$, $e^{0.5} = 1.65$, $e^{-0.25} = 0.78$, $e^{-0.5} = 0.61$.

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate $p_2(A|B)$, $p_2(B|B)$, $p_2(C|B)$ and $p_2(D|B)$.

Solution

ENG)

- a) Write these three sentences.
 - First-order proximity is the similarity between two nodes that are directly connected by an edge.
 - Second-order proximity is the similarity between two nodes that share many neighbours, even without an edge.
 - So LINE gives each node two vectors: its own embedding and its context embedding.
- b) $p_2(A|B) = 0.2963$, $p_2(B|B) = 0.3819$, $p_2(C|B) = 0.1412$, $p_2(D|B) = 0.1806$

11. (GCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 0 \\ 4 \\ 0 \\ -1 \\ -3 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how information is propagated in a GCN layer.
- Calculate $\tilde{A}$.
- Calculate the output of the GCN model at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - In one layer each node aggregates the features of its direct neighbours and takes the average.
 - The row-normalized matrix $\tilde{A}$ equals $D^{-1}A$, which performs this averaging.
 - After k layers a node has received information from all nodes within k hops.
- $\tilde{A} = [0, 0.5, 0.5, 0, 0]; [0.3333, 0, 0, 0.3333, 0.3333]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0.5, 0.5, 0, 0]; [0, 0.5, 0.5, 0, 0]$
- $H_1 = [2, 0, 0, 2, 2]$

12. (GraphSAGE) Consider an undirected graph G of five nodes given in the following figure. Each node has initial features that are the numbers standing next to it. According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

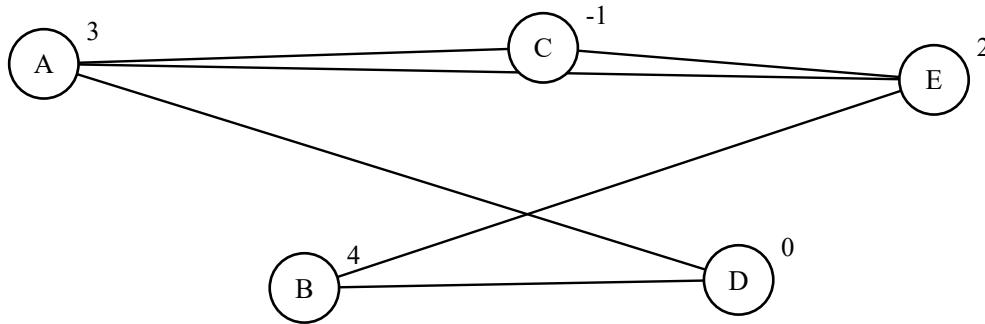


Figure. Graph G

- What is the main idea behind GraphSAGE, and how does it differ from traditional GCNs?
- Calculate the feature of each node at $k = 1$.
- Calculate a graph-level embedding h_G by using a 'Mean' global pooling when $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE learns an aggregator function instead of a fixed embedding for each node.
 - It samples a fixed number of neighbours, so it can be trained in mini-batches on very large graphs.
 - A GCN is transductive, but GraphSAGE is inductive, so it can embed a node that was never seen in training.
- $h(A) = [3, 0.3333]$, $h(B) = [4, 1]$, $h(C) = [0, 2.5]$, $h(D) = [0, 3.5]$, $h(E) = [2, 2]$
- $h_G = [1.8, 1.8667]$

13. (ClusterGCN) Consider an undirected graph G of six nodes given in the following figure. The graph G contains two clusters G_1 and G_2 . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

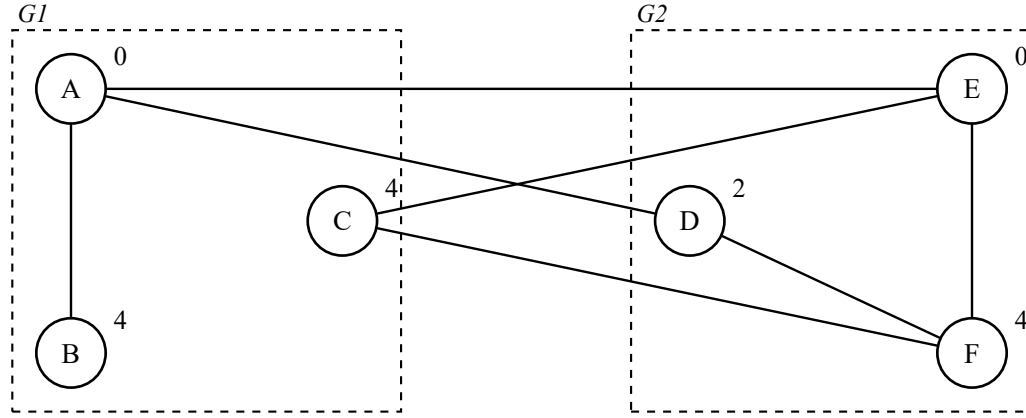


Figure. Graph G

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - Mini-batch training updates the parameters using a small subset of the data at a time.
 - A large graph does not fit in memory, so ClusterGCN partitions the graph into densely connected clusters.
 - A good partition keeps most edges inside a batch, so little information is lost at the boundary.
- $h(A) = [0, 4], h(B) = [4, 0], h(C) = [4, 0], h(D) = [2, 4], h(E) = [0, 4], h(F) = [4, 1]$

14. (GraphSAINT) Consider an undirected graph G of six nodes given in the following figure. The graph G has a sampled subgraph G_1 . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node i at layer k can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where $\parallel$ is a concatenation and $\text{ReLU}(x) = \max(0, x)$.

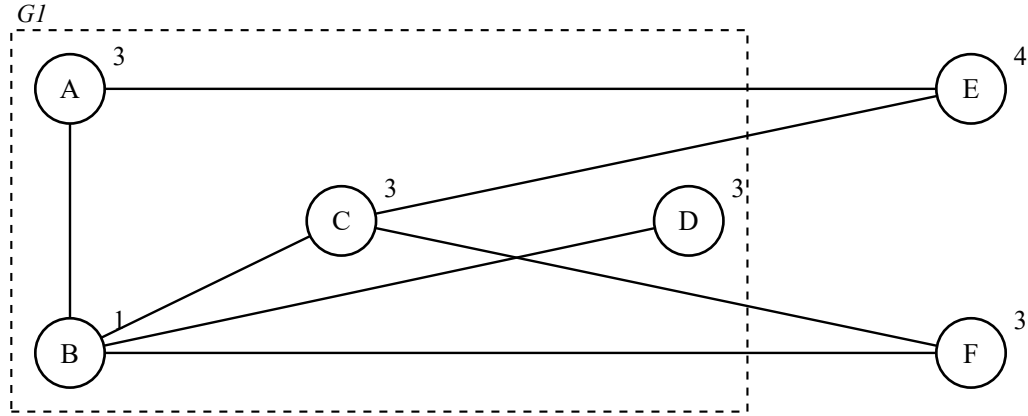


Figure. Graph G

- How does GraphSAINT's sampling differ from node-wise or layer-wise sampling?
- Calculate the output representations of nodes A, B, C and D at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - GraphSAGE and FastGCN sample while building the layers, so sampling is repeated at every layer.
 - GraphSAINT samples a subgraph first and then builds the full GNN on that subgraph.
 - So the cost does not grow with depth, and normalization coefficients correct the sampling bias.
- $h(A) = [3, 1]$, $h(B) = [1, 3]$, $h(C) = [3, 1]$, $h(D) = [3, 1]$

15. (JK Network) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -1 \\ 4 \\ 2 \\ -1 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \max(\sigma(\tilde{A} H^{(0)}), \sigma(\tilde{A} H^{(1)}), \dots, \sigma(\tilde{A} H^{(k-1)}))$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem.
- Calculate $\tilde{A}$ and $H^{(1)}$.
- Calculate the output representations at layer $k = 2$.

Solution

ENG)

- Write these three sentences.
 - In a deep GCN every node repeatedly averages its neighbours, so all representations become similar. This is over-smoothing.
 - JKNet keeps the representation of every layer and combines them at the end by max, concatenation or an LSTM.
 - The best depth is different for each node, so each node can select the depth that suits it.
- $\tilde{A} = [0, 0.3333, 0.3333, 0, 0.3333]; [0.5, 0, 0, 0, 0.5]; [0.3333, 0, 0, 0.3333, 0.3333]; [0, 0, 1, 0, 0]; [0.3333, 0.3333, 0.3333, 0, 0]$
 $H^1 = [0.6667, 0, 0.6667, 4, 1.3333]$
- $H^2 = [0.6667, 1, 2, 4, 1.3333]$

16. (GCNII) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -3 \\ 3 \\ 4 \\ -1 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer (k) can be calculated as:

$$H^{(k)} = \sigma[((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)})]$$

where $\tilde{A} = D^{-1}A$, I_n is the identity matrix, $\alpha = 0.5$, $\beta = 0.4$, and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- What are the two major components introduced in GCNII to overcome the depth limitation?
- Calculate $\tilde{A}$.
- Calculate the output representations at layer $k = 1$.

Solution

ENG)

- Write these three sentences.
 - The two components are the initial residual connection and the identity mapping.
 - The initial residual connection adds a fraction alpha of the input H zero back at every layer.
 - The identity mapping keeps the weight matrix close to the identity, so a deep model is not destroyed.
- $\tilde{A} = [0, 0, 0.5, 0.5, 0]; [0, 0, 0.5, 0, 0.5]; [0.3333, 0.3333, 0, 0.3333, 0]; [0.3333, 0, 0.3333, 0, 0.3333]; [0, 0.5, 0, 0.5, 0]$
- $H_1 = [1.35, 0, 1.1, 1.5, 0]$

17. (DeepGCN) Given a graph with an adjacency matrix A and initial node feature matrix $H^{(0)}$ as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -3 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection at layer (k) is:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where $\tilde{A} = D^{-1}A$ and σ is a ReLU function $\text{ReLU}(x) = \max(0, x)$.

- Explain why a residual connection allows a GCN to be made deeper.
- Calculate the output representations at layer $k = 1$ and $k = 2$.
- Compare $H^{(1)}$ with the output of a plain GCN without the residual connection.

Solution

ENG)

- Write these three sentences.
 - Deep GCNs suffer from vanishing gradients and over-smoothing, so adding layers makes the model worse.
 - A residual connection passes the input forward unchanged, so each layer only has to learn the change.
 - If no change is useful the layer can output zero, and gradients also flow back through the skip path.
- $H1 = [-2, 1, 2, 0]$
 $H2 = [-1, 1, 2, 0]$
- Plain GCN without residual: $H1 = [1, 0, 0, 0]$ (the node's own value and negatives disappear)